

Universidad Nacional Autónoma de México
Facultad de Ingeniería



Práctica No.04
Algoritmo de Cifrado Playfair

Asignatura:

CRIPTOGRAFÍA

Profesor: *Dr. Alfonso Francisco De Abiega L'Eglisse*

Grupo: 08

Integrantes:

Espinoza Matamoros Percival Ulises
García Cortés Adolfo de Jesús
Lugo Manzano Rodrigo
Montiel Juárez Oscar Iván

Semestre: 2027-1

1. Explicación del Método

El método usa una **matriz de 5×5** con 25 letras. Como el alfabeto tiene 26 letras, la `j` se cambia por la `i` y las dos comparten la misma celda.

1.1. Cómo se arma la matriz

Se escribe primero la palabra clave, sin repetir letras, y después se agregan las letras que faltan del alfabeto en orden. Con la clave `seguridad` la matriz queda así:

s	e	g	u	r
i	d	a	b	c
f	h	k	l	m
n	o	p	q	t
v	w	x	y	z

1.2. Preparar el mensaje

- Se quitan los espacios y la `j` se cambia por `i`.
- Se separa el mensaje en pares de letras.
- Si un par tiene dos letras iguales, se mete una `x` en medio (por ejemplo, `ll` pasa a `lx`).
- Si al final queda una letra sola, se completa con `x`. Si esa letra ya es `x`, se usa `q`.

1.3. Reglas para cifrar cada par

1. **Misma fila:** cada letra se cambia por la que está a su derecha.
2. **Misma columna:** cada letra se cambia por la que está debajo.
3. **Rectángulo:** cada letra se cambia por la que está en su misma fila, pero en la columna de la otra letra.

Para descifrar se hace lo mismo al revés: a la izquierda en vez de a la derecha, y arriba en vez de abajo. Las reglas del rectángulo no cambian.

Ejemplo con la clave `seguridad`: el par `ho` está en la misma columna (la `h` está en la fila 3 y la `o` en la fila 4), así que se toman las letras de abajo: `h`→`o` y `o`→`w`. El resultado es `ow`. El par `la` forma un rectángulo, así que `l`→`k` y `a`→`b`, y el resultado es `kb`.

2. Implementación en Python

El programa guarda la matriz como una cadena de 25 letras. Para saber la fila y la columna de una letra se usa `divmod(posición, 5)`.

2.1. Generación de la matriz

```python
def generar_matriz(clave):

   # Construye la matriz 5x5 (como cadena de 25 letras):
   # primero las letras de la clave sin repetir y después el resto del
      alfabeto.

    clave = clave.replace("j", "i")
    matriz = ""
    for caracter in clave + A_PLAYFAIR:
        if caracter in A_PLAYFAIR and caracter not in matriz:
            matriz += caracter
    return matriz


def mostrar_matriz(matriz):
    #Imprime la matriz de 5x5
    for i in range(5):
        print("   ", " ".join(matriz[i * 5:(i + 1) * 5]))
```

La función recorre la clave seguida del alfabeto de 25 letras (`A_PLAYFAIR`, que ya no tiene la `j`). Solo agrega una letra si todavía no está en la matriz. Así quedan primero las letras de la clave y después las demás.

2.2. Preparación del mensaje en digramas

```python
def preparar_digrafos(mensaje):

    #Quita espacios, cambia 'j' por 'i' y separa el mensaje en pares de
       letras.
    #- Si un par tiene dos letras iguales se inserta una 'x' entre ellas.
    #- Si queda una letra sola al final se completa con 'x'.

    texto = mensaje.replace(" ", "").replace("j", "i")
    pares = []
    i = 0
    while i < len(texto):
        a = texto[i]
        if i + 1 < len(texto) and texto[i + 1] != a:
            b = texto[i + 1]
            i += 2
        else:
            b = "x" if a != "x" else "q"   # relleno (si la letra ya es 'x
               ' se usa 'q')
            i += 1
        pares.append(a + b)
    return pares
```

Esta función quita los espacios, cambia la `j` por `i` y forma los pares. Si dos letras seguidas son iguales, o si queda una letra sola, usa la `x` como relleno.

2.3. Reglas de Playfair

```python
def transformar_par(par, matriz, direccion):

    #Aplica las reglas de Playfair a un par de letras.
    #direccion = 1  -> cifrar
    #direccion = -1 -> descifrar

    f1, c1 = divmod(matriz.index(par[0]), 5)
    f2, c2 = divmod(matriz.index(par[1]), 5)

    if f1 == f2:
        # Misma fila: se toma la letra a la derecha (cifrar) o a la
            izquierda (descifrar)
        n1 = matriz[f1 * 5 + (c1 + direccion) % 5]
        n2 = matriz[f2 * 5 + (c2 + direccion) % 5]
    elif c1 == c2:
        # Misma columna: se toma la letra de abajo (cifrar) o de arriba (
            descifrar)
        n1 = matriz[((f1 + direccion) % 5) * 5 + c1]
        n2 = matriz[((f2 + direccion) % 5) * 5 + c2]
    else:
        # Rectángulo: se intercambian las columnas
        n1 = matriz[f1 * 5 + c2]
        n2 = matriz[f2 * 5 + c1]

    return n1 + n2
```

El parámetro `direccion` vale 1 para cifrar y -1 para descifrar. Con el operador `%` de módulo 5 la matriz "da la vuelta": si una letra está en el borde derecho, la siguiente es la primera de esa fila.

2.4. Funciones de cifrado y descifrado

```python
def cifrar(mensaje, clave, mostrar_detalle=False):
    matriz = generar_matriz(clave)
    mensaje_cifrado = ""

    for par in preparar_digrafos(mensaje):
        cifrado = transformar_par(par, matriz, 1)
        if mostrar_detalle:
            print(f"'{par}' -> '{cifrado}'")
        mensaje_cifrado += cifrado

    return mensaje_cifrado


def descifrar(mensaje_cifrado, clave, mostrar_detalle=False):
    matriz = generar_matriz(clave)
    mensaje_descifrado = ""

    for i in range(0, len(mensaje_cifrado), 2):
        par = mensaje_cifrado[i:i + 2]
```

```
        original = transformar_par(par, matriz, -1)
        if mostrar_detalle:
            print(f"'{par}' -> '{original}'")
        mensaje_descifrado += original

    return mensaje_descifrado
```

Ambas funciones usan la misma matriz. La de cifrado recorre los digramas preparados; la de descifrado recorre el texto cifrado de dos en dos letras.

2.5. Validación de datos

El programa pide el mensaje y la clave. Solo acepta letras minúsculas de la `a` a la `z` (sin números ni símbolos) y la clave no puede estar vacía:

```
# Validación de la clave (solo letras minúsculas, sin espacios)
while True:
    clave = input("> Ingrese la palabra clave (solo letras a-z): ").lower
        ()

    es_valida = True
    for caracter in clave:
        if caracter not in A:
            es_valida = False
            break

    if es_valida and len(clave) > 0:
        break
    else:
        print("\n![Error]: La clave debe contener solo letras minúsculas (
            a-z), sin espacios ni otros caracteres.\n")

# Mostrar la matriz generada
print("\n--- Matriz Playfair 5x5 (i y j comparten celda) ---")
mostrar_matriz(generar_matriz(clave))
```

3. Pruebas del Programa

3.1. Prueba 1: mensaje “hola equipo diez”, clave `seguridad`

En la Figura 1 se ve la matriz que genera el programa, el cifrado de cada par y el descifrado. El mensaje cifrado es `owkbuosbqpadrw`.

```
 Algoritmo de Cifrado Playfair

> Ingrese el mensaje a cifrar: hola equipo diez
> Ingrese la palabra clave (solo letras a-z): seguridad

--- Matriz Playfair 5x5 (i y j comparten celda) ---
    s e g u r
    i d a b c
    f h k l m
    n o p q t
    v w x y z

--- Cálculo del cifrado (par -> par cifrado) ---
'ho' -> 'ow'
'la' -> 'kb'
'eq' -> 'uo'
'ui' -> 'sb'
'po' -> 'qp'
'di' -> 'ad'
'ez' -> 'rw'

--- Cálculo del descifrado (par cifrado -> par) ---
'ow' -> 'ho'
'kb' -> 'la'
'uo' -> 'eq'
'sb' -> 'ui'
'qp' -> 'po'
'ad' -> 'di'
'rw' -> 'ez'

------------- Resultados -------------

--> Clave utilizada:           seguridad
--> Mensaje original:          hola equipo diez
--> Mensaje cifrado:           owkbuosbqpadrw
--> Mensaje descifrado:        holaequipodiez

---------------------------------------
```

Figura 1: Prueba 1

Al descifrar se obtiene `holaequipodiez`. Es el mismo mensaje, pero sin espacios, porque el algoritmo los quita antes de cifrar.

3.2. Prueba 2: letras repetidas

Para comprobar la regla de las letras iguales se cifró “hello world” con la clave `monarchy`. En el par `ll` el programa mete una `x` en medio.

```
 Algoritmo de Cifrado Playfair

> Ingrese el mensaje a cifrar: hello wolrd
> Ingrese la palabra clave (solo letras a-z): monarchy

--- Matriz Playfair 5x5 (i y j comparten celda) ---
    m o n a r
    c h y b d
    e f g i k
    l p q s t
    u v w x z

--- Cálculo del cifrado (par -> par cifrado) ---
'he' -> 'cf'
'lx' -> 'su'
'lo' -> 'pm'
'wo' -> 'vn'
'lr' -> 'tm'
'dx' -> 'bz'

--- Cálculo del descifrado (par cifrado -> par) ---
'cf' -> 'he'
'su' -> 'lx'
'pm' -> 'lo'
'vn' -> 'wo'
'tm' -> 'lr'
'bz' -> 'dx'

------------- Resultados -------------

--> Clave utilizada:           monarchy
--> Mensaje original:          hello wolrd
--> Mensaje cifrado:           cfsupmvntmbz
--> Mensaje descifrado:        helxlowolrdx

----------------------------------------
```

Figura 2: Prueba 2

Aquí se nota que el mensaje descifrado trae una `x` extra y no es exactamente igual al original. Esto es normal en Playfair: la `x` de relleno se queda en el texto y quien lee el mensaje la ignora.

4. Repositorio

El código de esta práctica 04 se puede consultar en el siguiente repositorio:

https://github.com/percivalespi/Criptografia/blob/main/practica4/codigo/playfair.py

5. Conclusión

En esta práctica aprendimos que Playfair cifra las letras en pares, en lugar de hacerlo una por una, por lo que es más seguro que cifrados sencillos como el César. Lo que más trabajo nos costó fue manejar algunos casos especiales, como cuando la `j` se junta con la `i`, cuando se repite una letra en el mismo par o cuando queda una sola letra al final. Usar `divmod` para obtener la fila y la columna, y el módulo 5 para recorrer la matriz, nos ayudó a escribir las reglas de forma sencilla. También vimos que, al descifrar un mensaje, pueden aparecer letras adicionales como la `x`, por lo que es importante revisar el resultado.